

Assignment 1: Multilayer Perceptrons, Loss Functions, and Optimizers

Deep Learning (Graduate Level)

Fall 2025

Overview

In this first assignment, you will build and train a **Multilayer Perceptron (MLP)** in **PyTorch** to classify biomedical images from the **MedMNIST dataset**. Through this exercise, you will deepen your understanding of neural network design, activation functions, loss functions, and optimization algorithms.

Dataset

We will use the **MedMNIST** dataset, a collection of small biomedical image datasets standardized for educational and benchmarking purposes.

- Dataset link: MedMNIST on Kaggle
- Each sub-dataset (e.g., PathMNIST, OrganMNIST, BloodMNIST) contains 28×28 RGB or grayscale images.
- For this assignment, select **one dataset** (e.g., PathMNIST or BloodMNIST). You may optionally explore multiple datasets for extra credit.

Learning Objectives

By completing this assignment, you will:

- Implement an MLP from scratch in PyTorch.
- Experiment with different activation functions, loss functions, and optimizers.
- Understand the effect of network depth, width, and regularization.
- Develop systematic experimentation and reporting skills.

Part 1: Data Preparation

- a) Download one MedMNIST subset from Kaggle.
- b) Normalize the input features to have zero mean and unit variance.
- c) Split the dataset into training, validation, and test sets (e.g., 70/15/15).

Part 2: MLP Implementation

- a) Implement an MLP with at least one hidden layer (e.g., 128–64–number of classes).
- b) Use `ReLU` activation initially.
- c) Apply appropriate weight initialization (e.g., Xavier or Kaiming).
- d) Train the model using `CrossEntropyLoss`.

Part 3: Experiments and Analysis

Design and run a series of controlled experiments. Each experiment should include: - A short motivation (why you test this) - Clear configuration (what you changed) - Results table and brief analysis (what you observed)

Experiment 1: Activation Functions

Compare the performance of different activation functions:

- `ReLU`
- `LeakyReLU`
- `Tanh`
- `Sigmoid`

Report training time, validation accuracy, and convergence behavior.

Experiment 2: Optimizers

Compare at least three optimizers:

- `SGD` (with and without momentum)
- `Adam`
- `RMSProp`

Discuss how learning rate and momentum parameters affect convergence and final accuracy.

Experiment 3: Network Depth and Width

Construct and compare models with:

- Shallow (1 hidden layer)
- Medium (3 hidden layers)
- Deep (5+ hidden layers)

Plot training and validation accuracy curves, and analyze overfitting tendencies.

Experiment 4: Regularization Techniques

Implement:

- Dropout (e.g., 0.2, 0.5)
- L2 weight decay
- Batch normalization

Study how each technique affects generalization performance.

Experiment 5: Learning Rate Scheduling

Try at least two learning rate schedulers (e.g., StepLR, CosineAnnealingLR, ReduceLROnPlateau). Observe how they influence convergence stability and training speed.

Experiment 6: Loss Functions (Optional)

Try alternative loss functions (e.g., `MSELoss`, `NLLLoss`) and explain why some may be unsuitable for classification.

Part 4: Reporting

Your report (PDF) should include:

- A short description of your chosen dataset (size, number of classes, image examples)
- Implementation details (network architecture, optimizer parameters, etc.)
- Experimental results in table or plot form
- Key insights from each experiment
- A short discussion of what worked best and why

Submission Instructions

- Submit a Jupyter Notebook (.ipynb) with clean, documented code.
- Submit a report (.pdf) summarizing your findings.
- Include plots of loss/accuracy curves and comparative tables.

Grading (100 points total)

Data preparation and preprocessing	10 pts
MLP implementation	15 pts
Experiments (activation, optimizer, etc.)	50 pts
Analysis and insights	15 pts
Report clarity and visualizations	10 pts

Extra Credit (up to +10 pts)

- Try another MedMNIST subset and compare performance.
- Implement early stopping and show its impact.
- Use mixed-precision training or learning rate warmup.

Policy on the Use of Large Language Models (LLMs)

In this course, we emphasize the development of a **fundamental understanding of deep learning principles and implementation**. While Large Language Models (LLMs) such as ChatGPT, Claude, or Copilot can be helpful tools for general guidance, **you are not permitted to use them to generate or write any part of your assignment solution** at this stage.

This restriction is in place because:

- The goal of this assignment is to strengthen your ability to design, implement, and debug models from first principles.
- Using LLMs at this early stage can prevent you from developing the deep intuition required for later, more complex assignments involving custom architectures and advanced optimization techniques.
- Students who rely on LLM-generated code or explanations without understanding them often struggle significantly in later projects, where critical reasoning and independent implementation are required.

Therefore, any use of LLMs to produce code, analysis, or written explanations for this assignment is prohibited. You may, however, use documentation such as the official PyTorch library reference or scientific papers for conceptual understanding.

Violations of this policy will be treated as academic misconduct. More importantly, students who bypass the learning process at this stage will face severe difficulties in subsequent assignments that require originality and deeper understanding.